GUIA VISUAL 053

ASPAS E EXPANSÕES

053
GUIAS LINUX

*O script de limpeza tinha uma linha só e rodou oito meses sem incidente — até o dia em que `DESTINO` passou a valer `/srv/Arquivos Antigos`. Nenhum comando estava errado. Faltavam **duas aspas**.*

A LINHA QUE VOCÊ DIGITA NÃO É A LINHA QUE RODA

A LINHA DIGITADA	`cp ~/dados{1,2}.csv "$DEST"/$(date +%F)`
1 · CHAVES	`cp ~/dados1.csv ~/dados2.csv "$DEST"/$(date +%F)`
2 · TIL	`cp /home/kali/dados1.csv … "$DEST"/$(date +%F)`
3 · VARIÁVEIS, $(( )) E $( )	`cp /home/kali/dados1.csv … "/srv/Back up"/2026-08-03`
4 · DIVISÃO EM PALAVRAS	`[cp] [/home/kali/dados1.csv] [/home/kali/dados2.csv] [/srv/Back up/2026-08-03]`
5 · GLOBBING	`sem curinga nesta linha: nada muda`
6 · REMOÇÃO DAS ASPAS	`as aspas somem — elas nunca chegam ao comando. O` `cp` `recebe 3 argumentos.`

SEM ASPAS
`rm -rf $DESTINO/*`
`rm` `-rf` `/srv/Arquivos` `Antigos/*` → o 2º caminho é resolvido a partir do **diretório atual**

COM ASPAS
`rm -rf "$DESTINO"/*`
`rm` `-rf` `/srv/Arquivos Antigos/relatorio.pdf` → um argumento por arquivo real

Ensaie todo `rm -rf` com `echo` na frente. `echo rm -rf "$DESTINO"/*` mostra exatamente os argumentos que o comando receberia, sem apagar nada. O `/*` fica **fora** das aspas de propósito: é ele que precisa da etapa 5 para virar a lista de arquivos. **Aspas em volta do caminho, curinga fora** — é o que separa o *dado* (o valor da variável) da *sintaxe* (o curinga que você escreveu).

As etapas **4 e 5 são as perigosas**, e as duas acontecem **depois** de a variável virar valor. O shell não distingue o espaço que você digitou daquele que veio de dentro do valor — para ele é tudo a mesma linha. Aspas duplas não impedem a substituição da etapa 3; impedem que o resultado dela seja fatiado na 4 ou reinterpretado como curinga na 5.

AS FORMAS DE CITAR · E AS OUTRAS EXPANSÕES

FORMA	O QUE DESLIGA	O QUE SOBRA LIGADO
sem aspas	nada	tudo — inclusive a divisão e o curinga
`'simples'`	**tudo**	nada: cada byte é literal
`"duplas"`	divisão em palavras e *globbing*	`$var`, `$(cmd)`, `$((..))`, `\`
`\c`	só o próximo caractere	o resto da linha
`$'texto'`	como as simples, mas…	interpreta `\n`, `\t`, `\x41`

`$(( a + b ))`
Aritmética **só inteira**: `10/4` vale **2**. Decimal é trabalho de `bc`, `awk` ou `python3`.

`{1..5} · {a..e} · {0..20..5}`
Primeira etapa, logo **não enxerga variável**: `{1..$n}` sai literal — use `seq 1 "$n"`. Idioma clássico: `cp conf{,.bak}`.

`~ · ~root · ~-`
Não expande dentro de aspas: `"~/dados"` é literal. Em script, `"$HOME/dados"`.

`<(comando)`
Entrega a saída como se fosse arquivo: `diff <(sort a) <(sort b)`. E `while … done < <(cmd)` é o laço **sem** subshell.

O `eval` transforma dado em sintaxe — é a via curta para injeção de comando. Com `entrada="/tmp; rm -rf ~"`, o `eval "ls $entrada"` executa também o `rm`: o ponto e vírgula que estava no *dado* virou sintaxe. Defesas, em ordem: **1.** não use `eval` — quase sempre há array (`"${opcoes[@]}"`), `${!var}` ou `case` · **2.** valide por **lista de permitidos**, nunca removendo caracteres proibidos · **3.** passe como argumento e encerre as opções: `comando -- "$valor"` · **4.** `printf %q` quando montar linha for inevitável. O mesmo vale para `bash -c "$linha"`, `find -exec sh -c` e, sobretudo, `ssh host "cmd $VAR"` — que atravessa **dois** shells: o local expande, o remoto reinterpreta o resultado inteiro.